Environment and Planning A, 1977, volume 9, pages 1423-1428

Algorithm 3: A procedure to generate pseudo-random aggregations of *N* zones into *M* zones, where *M* is less than *N*

S Openshaw
Department of Town and Country Planning, Newcastle University, Newcastle upon Tyne, NE1 7RU, England

Referee R S Baxter
Building Research Station, Garston, Watford, Herts WD2 7JR, England
Received 26th January 1977, in revised form 25th March 1977

Language Subroutine ZONE1 is written in ANSI Fortran

Description and purpose

The purpose of this subroutine is to generate a pseudo-random aggregation of N zones into M zones, where M is less than N, such that each of the original N zones is allocated to one of the M zones, and such that all the N-zone members of each M zone are spatially contiguous. Successive calls of the subroutine can be used to build up a random sample of alternative zoning systems from the population of all possible zoning systems with M zones. The distributional characteristics of various statistics calculated for a sample of randomly generated zoning systems can be used to investigate various zoning-system effects. The subroutine can also be used for simulation experiments concerned with investigating the properties of spatially aggregated data under different data assumptions, and as a means of generating unbiased starting aggregations for redistricting and zone-design procedures.

The algorithm is described in full in an earlier paper of mine (Openshaw, 1977, page 171), and is summarized below in simpler terms. The method was developed from a bipartisan districting algorithm described by Taylor (1973). This is rewritten to incorporate a greater degree of randomization, and then generalized to serve as a random partition generator.

A study region is assumed to be completely covered by N zones numbered 1 to N. Usually these zones are the product of a prior aggregation of the data. A contiguity matrix is assumed to be available, describing in full the spatial adjacency of these zones. A random aggregation of these N original zones into M new zones is required.

The first task is to select at random M of the original zones to act as core zones for the aggregation. These core zones are numbered 1 to M. Now select at random any one of these core zones and form a list of unallocated original zones which are adjacent to the chosen zone. From this list select at random any original zone and allocate it to the chosen core zone. This process, starting with another random selection of a core zone from the list of M zones, is repeated until either all of the original zones are allocated or until no unallocated zone remains which is contiguous to any of the M zones. In the latter instance not all of the N original zones will have been allocated because of islands of contiguities or of errors within the contiguity matrix. This situation may be rectified by recalling the subroutine, and thus trying a different set of core zones, or by changing the contiguity matrix.

Structure

Subroutine ZONE1 (IFXZNE, IP, NUM, IRN, N, M, IFXDIM, IRNDIM, IW3DIM, IFAULT, IPART, IWORK1, IWORK2, IWORK3)

Arguments

All arguments listed below are in accordance with the usual Fortran naming conventions.

Input and output

IFXZNE An integer array containing IFXDIM−1 values in the range 1 to N; it defines external or fixed zones which are not affected by the operation of the subroutine. Each of the original zones included in this list becomes a separate zone in the output aggregation.

IP An integer array of N elements containing the subscript locations in IRN of the first entry in the contiguity list for each of the original zones.

NUM An integer array of N elements containing the subscript locations in IRN of the last entry in the contiguity list for each of the original zones. The calling program must ensure that the pointer values contained in IP and NUM are correct.

IRN An integer array containing the full contiguity matrix stored as a chained list. Reciprocal contiguities must be included; for example, if zone 1 is recorded as contiguous to zone 76 then zone 76 must include zone 1 in its list of contiguities. IRN is accessed via IP and NUM. The contiguity list for any zone I will be found in consecutive subscript locations in IRN starting at IP(I) and ending at NUM(I). If for some reason any zone J has no contiguities assigned to it, then set NUM(J) less than IP(J). Contiguities need not be provided for any zones included in IFXZNE.

N An integer constant set to the original number of zones.

M An integer constant set to the desired number of zones in the randomly generated zoning system. M must be less than N. The value assigned to M includes any zones declared as fixed in IFXZNE.

IFXDIM An integer constant specifying the size of the array IFXZNE. The value assigned to IFXDIM is the number of entries in IFXZNE plus one.

IRNDIM An integer constant set to the number of elements in IRN. This must equal or exceed the value contained in NUM(N).

IW3DIM An integer constant specifying the size of array IWORK3. The maximum size is N−M but a smaller value may be used.

Output only

IFAULT An integer error indicator.

IFAULT = 0 No errors are detected.

> 0 Gives the number of original zones which are unallocated due to contiguity islands or errors in the contiguity matrix. The zoning-system output in IPART will now contain M+IFAULT zones. A list of these unallocated zones is output in IWORK1.

= −1 Will not occur unless there are addressing errors resulting from an incorrect declaration of one or more of the arguments.

= −2 The value assigned to IFXDIM exceeds M.

= −3 The value assigned to IW3DIM is set too small.

= −4 The value assigned to N is less than or equal to the value assigned to M.

IPART An integer array of N elements containing the randomly generated zoning system when IFAULT = 0 or IFAULT > 0; otherwise this array is undefined. The zoning system is described as a classification array so that any original zone I is allocated to new zone IPART(I). The values in IPART range from 1 to M or exceptionally from 1 to M+IFAULT.

IWORK1 An integer array of N elements used as a workspace; however, when IFAULT > 0 it is used to return a list of IFAULT unallocated original zones.

Workspaces
IWORK1 An integer array of N elements used as a workspace (except when IFAULT > 0—see output listings).
IWORK2 An integer array of N elements used as a workspace.
IWORK3 An integer array of IW3DIM elements used as a workspace.

Auxiliary algorithm
A user-supplied function subroutine IRAND1 must be provided to generate a random integer in the range 1 to N. This subroutine is of the form K = IRAND1(N), where IRAND1 is assigned the value of a random integer between 1 and N.

Time
The time taken will depend on the values assigned to N, M, and IRNDIM. For example, when N = 99, M = 6, and IRNDIM = 442 it is possible to generate approximately 12 random zoning systems per second of central processing unit (CPU) time on an IBM 370/168 running under MTS with a Fortran (H) compilation. When N = 1219, M = 30, and IRNDIM = 8316 then the time increases to one zoning system every 10 seconds of CPU time. These times are based on a version of IRAND1 which uses a Harwell library subroutine to generate random integers.

Restrictions
There is no guarantee that all the aggregations will be unique in a large sample of random aggregations of N zones into M zones. The zoning systems generated correspond to sampling with replacement. The algorithm will not generate all possible aggregations of N zones into M zones, only a random sample drawn from this population.

References
Openshaw S, 1977 "Optimal zoning systems for spatial interaction models" *Environment and Planning A* **9** 169-184
Taylor P J, 1973 "Some implications of the spatial organisation of elections" *Transactions of the Institute of British Geographers* **60** 121-136

Subroutine ZONE1

```
      SUBROUTINE ZONE1(IFXZNE,IP,NUM,IRN,N,M,IFXDIM,IRNDIM,IW3DIM,
     1IFAULT,IPART,IWORK1,IWORK2,IWORK3)
C
C        THIS SUBROUTINE GENERATES A PSEUDO-RANDOM AGGREGATION
C        OF N ZONES INTO M ZONES, WHERE M < N.
C
C        ARGUMENTS
C        ON ENTRY
C        IFXZNE       INTEGER VECTOR (IFXDIM) CONTAINING FIXED ZONES
C        IP           INTEGER VECTOR (N) CONTAINING START POINTERS
C        NUM          INTEGER VECTOR (N) CONTAINING END POINTERS
C        IRN          INTEGER VECTOR (IRNDIM) CONTAINING CONTIGUITY MATRIX
C        N            NUMBER OF ORIGINAL ZONES
C        M            NUMBER OF ZONES IN OUTPUT AGGREGATION
C        IFXDIM       NUMBER OF FIXED ZONES PLUS ONE
C        IRNDIM       DIMENSION OF IRN
C        IW3DIM       DIMENSION OF IWORK3
C
C        ON RETURN
C        IFAULT       IS A FAULT NUMBER
C        IPART        INTEGER VECTOR (N) CONTAINING ZONING SYSTEM
C                     IF IFAULT .GE. 0, OTHERWISE UNDEFINED
C        IWORK1       INTEGER VECTOR (N) CONTAINING LIST OF ZONES
C                     IF IFAULT .GT. 0, OTHERWISE UNDEFINED
C
C        WORKSPACE
C        IWORK1       INTEGER VECTOR (N)
C        IWORK2       INTEGER VECTOR (N)
C        IWORK3       INTEGER VECTOR (IW3DIM)
C
C        FAULTS
C        IF IFAULT=0     NORMAL TERMINATION
C                 >0     THEN M+IFAULT ZONES IN IPART, DUE TO CONTIGUITY ISLANDS
C                 =-1    IMPOSSIBLE EXIT UNLESS ADDRESSING ERRORS HAVE OCCURRED
C                 =-2    IFXDIM EXCEEDS M
C                 =-3    IW3DIM SET TOO SMALL
C                 =-4    N .LT. M
C
C        OTHER ROUTINE
C        THE USER-SUPPLIED FUNCTION IRAND1 TAKES ONE INTEGER ARGUMENT N
C        AND WHEN CALLED IS SET TO A RANDOM INTEGER IN THE RANGE 1 TO N
C
      DIMENSION IFXZNE(IFXDIM),IP(N),NUM(N),IRN(IRNDIM),IPART(N),
     1IWORK1(N),IWORK2(N),IWORK3(IW3DIM)
C
C        CHECK NUMBER OF ZONES
C
      IF(N.LE.M)GO TO 17
      IF(IFXDIM.GT.M)GO TO 15
C
C        INITIALISATION
C
      IK=0
      IFAULT=0
      DO 1 I=1,N
      IWORK1(I)=0
    1 IPART(I)=0
C
C        PUT IN IPART THOSE ORIGINAL ZONES WHICH ARE FIXED
C
      NE=IFXDIM-1
      IF(NE.EQ.0)GO TO 3
      DO 2 I=1,NE
      K=IFXZNE(I)
    2 IPART(K)=I
C
C        SELECT (M-NE) ORIGINAL ZONES TO ACT AS ACTIVE ZONE CORES
C
    3 NCORE=0
      DO 5 I=IFXDIM,M
    4 K=IRAND1(N)
C
C        REJECT IF ORIGINAL ZONE K ALREADY ALLOCATED AS A ZONE CORE
C
      IF(IPART(K).NE.0)GO TO 4
C
C        ELSE PLACE K IN ACTIVE CORE LIST (IWORK2)
C
      NCORE=NCORE+1
      IPART(K)=I
```

```
    5 IWORK2(NCORE)=I
      NE=NCORE+NE
C
C        NOW RANDOMLY SELECT ANY CORE (ICORE) FROM THE ACTIVE CORE LIST
C
    6 K=IRAND1(NCORE)
      ICORE=IWORK2(K)
      ICOREK=K
C
C        MAKE A LIST (IWORK1) OF UNALLOCATED ORIGINAL ZONES WHICH ARE
C        CONTIGUOUS TO ANY MEMBER OF THIS ZONE (ICORE)
C
      IK=IK+1
      L=0
      DO 8 I=1,N
      IF(IPART(I).NE.ICORE)GO TO 8
      K1=IP(I)
      K2=NUM(I)
      IF(K2.LT.K1)GO TO 8
      DO 7 J=K1,K2
      KR=IRN(J)
      IF(IPART(KR).NE.0)GO TO 7
      IF(IWORK1(KR).EQ.IK)GO TO 7
      IWORK1(KR)=IK
      L=L+1
      IF(L.GT.IW3DIM)GO TO 16
      IWORK3(L)=KR
    7 CONTINUE
    8 CONTINUE
C
C        IF THIS LIST (IWORK3) IS EMPTY THEN REMOVE THIS CORE (ICORE) FROM
C        THE ACTIVE CORE LIST (IWORK2)
C
      IF(L.EQ.0)GO TO 9
C
C        ELSE RANDOMLY SELECT ANY ORIGINAL ZONE FROM THIS LIST AND ALLOCATE
C        TO ZONE ICORE
C
      J=IRAND1(L)
      K=IWORK3(J)
      IPART(K)=ICORE
      NE=NE+1
      GO TO 6
C
C        REMOVE THIS ZONE CORE (ICORE) FROM ACTIVE CORE LIST
C
    9 IF(ICOREK.EQ.NCORE)GO TO 11
      J=ICOREK+1
      DO 10 I=J,NCORE
      K1=I-1
   10 IWORK2(K1)=IWORK2(I)
   11 NCORE=NCORE-1
C
C        IF REMAINING ACTIVE CORE LIST IS NOT EMPTY GO BACK AROUND
C
      IF(NCORE.GT.0)GO TO 6
C
C        CHECK WHETHER ALL THE ORIGINAL ZONES HAVE BEEN ALLOCATED
C
      IF(NE.EQ.N)GO TO 13
C
C        NOT ALL OF THE ZONES HAVE BEEN ALLOCATED
C
      K1=IFAULT+M
      DO 12 I=1,N
      IF(IPART(I).NE.0)GO TO 12
      K1=K1+1
      IPART(I)=K1
C
C        PUT UNALLOCATED CORE IN CORE LIST
C
      K1=K1+1
      IWORK2(1)=K1
      NCORE=1
      IFAULT=IFAULT+1
      GO TO 6
```

```
   12 CONTINUE
C
C        IMPOSSIBLE EXIT UNLESS THERE ARE ADDRESSING ERRORS
C
      IFAULT=-1
      RETURN
C
C        NORMAL EXIT
C
   13 IF(IFAULT.EQ.0)RETURN
C
C        FINAL PARTITION HAS MORE THAN M ZONES, SO SET IWORK1
C
      L=0
      DO 14 I=1,N
      IF(IPART(I).LE.M)GO TO 14
      L=L+1
      IWORK1(L)=I
   14 CONTINUE
      RETURN
   15 IFAULT=-2
      RETURN
   16 IFAULT=-3
      RETURN
   17 IFAULT=-4
      RETURN
      END
```

© 1977 a Pion publication printed in Great Britain